

A Comparative Study of Hash Map and Trie Data Structures for Efficient Movie Search Using MovieLens Dataset

Continuous Assignment 2 (15%)

Submission date: Sunday 3rd May 2026 @ 8pm

Overview:

In this assignment, you will:

- Implement and compare two fundamental data structures for efficient movie search:
 - Hash Map (for exact lookups)
 - Prefix Tree / Trie (for partial and autocomplete search)
- Apply these structures to a real-world movie dataset (MovieLens dataset from GroupLens Research), containing:
 - Movie ID
 - Title
 - Genres
 - Ratings (aggregated or derived)
- Design and implement an experimental framework to evaluate and benchmark:
 - Exact search performance (Hash Map vs Trie traversal)
 - Prefix-based search performance (Trie vs linear scan / alternative approaches)
 - Insertion performance for building each data structure
- Analyse and compare results in relation to theoretical time complexity, including:
 - $O(1)$ average-case lookup for Hash Maps
 - $O(k)$ prefix search complexity for Tries (where k is the length of the search prefix)
 - $O(n)$ linear scan baseline for comparison
 - Trade-offs in memory usage and preprocessing cost between structures

Source: csv Movie data file is available on the grouplens website

<https://grouplens.org/datasets/movielens/>

Learning Outcomes:

By completing this assignment, students will be able to:

- Implement and evaluate core data structures for efficient data retrieval, including:
 - Hash Maps (for constant-time exact lookup)
 - Prefix Trees / Tries (for efficient prefix-based search and autocomplete functionality)
- Apply appropriate data structures strategically to solve different types of search problems, including:
 - Exact match queries (Hash Map)
 - Partial / prefix-based queries (Trie)
 - Baseline linear search for comparison
- Design and implement an experimental framework to benchmark data structure performance using a real-world dataset.
- Measure and compare the practical runtime performance of different operations, including:
 - Movie insertion into data structures
 - Exact search operations
 - Prefix-based search operations
 - Retrieval of multiple matching results
- Relate empirical performance results to theoretical time complexity, including:
 - $O(1)$ average-case lookup for Hash Maps
 - $O(k)$ prefix search complexity in Tries (where k is the length of the query string)
 - $O(n)$ performance of linear scan-based search approaches
- Analyse the trade-offs between different data structures in terms of:
 - Time efficiency
 - Memory usage
 - Scalability with increasing dataset size
- Present findings in a structured technical report, including:
 - Experimental methodology
 - Performance comparison tables/graphs
 - Critical discussion of observed results
 - Justified conclusions supported by empirical evidence

The Movie Dataset:

The dataset used in this assignment is the MovieLens dataset provided by GroupLens Research, a research group within the University of Minnesota.

Source: <https://grouplens.org/datasets/movielens/>

The dataset contains:

- Movie ID (unique identifier)
- Title (including release year)
- Genres (one or more categories per movie)

Assignment Brief:

Your task is to implement and compare the performance of two core data structures used for efficient search and retrieval:

- Hash Map (for exact movie lookup)
- Prefix Tree / Trie (for prefix-based search and autocomplete functionality)

You will evaluate their efficiency when performing common operations on the movie dataset, including:

- Searching for movies by exact title or ID
- Performing prefix-based searches on movie titles
- Inserting movie records into each data structure
- Retrieving multiple matching results for partial queries

The performance of each data structure will be analysed and compared against their theoretical time complexities, such as:

- $O(1)$ average-case time complexity for Hash Map lookups
- $O(k)$ time complexity for Trie-based prefix searches (where k is the length of the search string)
- $O(n)$ linear search complexity as a baseline comparison

Additionally, you will design and conduct a set of controlled experiments to measure and compare runtime performance across different query types and dataset operations.

The goal of this assignment is to develop an understanding of how different data structures impact search efficiency in real-world applications, and how theoretical complexity compares with empirical performance when applied to a realistic dataset.

You are required to:

- 1. Load the movie dataset from a CSV file (MovieLens dataset from GroupLens Research), and extract relevant fields such as:**
 - Movie ID, Title, Genres
- 2. Implement and manage the dataset using the following data structures:**
 - Hash Map (for efficient exact lookup of movies)
 - Prefix Tree / Trie (for efficient prefix-based search and autocomplete functionality)
- 3. Support efficient operations on the dataset, including:**
 - Searching for movies by exact title or ID (Hash Map)
 - Performing prefix-based search on movie titles (Trie)
 - Inserting new movie records into both data structures
 - Retrieving multiple matching results for partial queries
- 4. Ensure that all data structure operations are:**
 - Implemented from scratch (no built-in hash map wrappers or trie libraries for core logic)
 - Designed to clearly demonstrate internal working of each structure
 - Applied to the same dataset to ensure a fair comparison of performance
- 5. Measure execution time for each operation:**
 - Insertion time for building each structure
 - Exact search time
 - Prefix search time (autocomplete queries)
- 6. Compare and analyse the results, discussing:**
 - Observed execution times across different operations
 - Performance differences between Hash Map and Trie
 - Trade-offs between exact lookup efficiency and prefix search capability
 - Relationship between empirical results and theoretical time complexity:
 - $O(1)$ average-case lookup for Hash Map
 - $O(k)$ prefix search complexity for Trie
 - $O(n)$ linear scan baseline (optional comparison)
- 7. Present findings in a structured technical report, including:**
 - Tables and/or graphs of performance results
 - Clear comparisons between data structures
 - Justified conclusions based on experimental evidence
 - Discussion of scalability and real-world application relevance

Experimental Requirements

For a structured evaluation, you should:

- Run each data structure (Hash Map and Trie) on multiple subsets of the MovieLens dataset, using different dataset sizes (movie records).
- Record the dataset size (number of movies) for each experiment.
- Measure execution time using a high-resolution timer (e.g., `time.perf_counter()`), for the following operations:
 - Insertion of movie records into each data structure
 - Exact search operations (Hash Map lookup vs linear baseline if included)
 - Prefix-based search operations (Trie autocomplete functionality)
- Repeat each experiment multiple times and calculate the average runtime to improve accuracy and reduce noise in results.
- Present results in clear tabular and/or graphical format for comparison between data structures and operations.

Analysis Component:

Your written analysis should address:

- Why a Hash Map provides efficient average-case performance for exact movie lookups, typically achieving $O(1)$ time complexity, and why this makes it suitable for large-scale datasets.
- Why a Trie (Prefix Tree) is more efficient for prefix-based searching of movie titles compared to linear search or hash-based approaches, achieving $O(k)$ time complexity where k is the length of the search string.
- Whether the empirical results obtained from your experiments align with the theoretical time complexities of:
 - Hash Map (exact search performance)
 - Trie (prefix-based search performance)
 - Linear scan baseline (if included)
- How the structure and size of the input dataset (e.g., 1K, 5K, 10K+ movie records) impacts:
 - Search performance for exact queries

- Prefix-based search efficiency
 - Data structure build/insertion time
 - Overall scalability of each approach
- The effectiveness of Hash Maps for fast retrieval of movie records using exact keys such as:
 - Movie ID
 - Full movie titleand why this approach becomes inefficient for partial or prefix-based queries.
- The effectiveness of Trie traversal as a method for implementing autocomplete functionality on movie titles, and how it compares to expected theoretical performance.

Submission:

The end deliverable should include:

- Well-structured Python source code.
- Benchmark results.
- A short analytical report discussing findings.

Grading Rubric

Component	100%–85%	84%–70%	69%–60%	59%–50%	49%–40%	Less than 39%
Data Model & Abstraction (10%)	Excellent design of Movie, HashMap, and Trie classes. Full encapsulation. Clean separation of exact and prefix search.	Very good design with full encapsulation. Minor issues in structure or separation of concerns.	Good design with basic encapsulation. Some minor design or abstraction issues.	Basic classes implemented. Functional but limited abstraction.	Weak design with poor structure and minimal abstraction..	Minimal or incorrect attempt at modelling movie data and structures.
Algorithm Implementations (30%)	All required structures (Hash Map, Trie) fully implemented from scratch. Correct logic, efficient design, and clean separation of exact and prefix search. No built-in search structures used.	All required structures implemented correctly with minor issues in logic or structure. No built-in structures used. Code is mostly clear and functional.	Most structures implemented correctly. Some inefficiencies or minor logical errors in search or insertion.	Basic implementation of Hash Map and/or Trie. Partial functionality or limited correctness.	One structure partially implemented with major issues or heavy inefficiencies.	Minimal or incorrect implementation. Heavy reliance on built-in structures or non-functional code.
Experimental Framework (25%)	Scientifically structured testing of Hash Map and Trie. Multiple dataset sizes used with systematic scaling. Repeated trials with averaged runtimes. High-resolution timing used. Clear tables of results.	Very good experimental design with multiple dataset sizes and correct timing. Some repetition of trials. Clear results with minor omissions.	Reasonable structure with more than one dataset size tested. Timing correct but limited repetition or analysis depth.	Basic timing tests with limited dataset scaling or repetition. Results present but lack depth.	Minimal testing with single dataset size or weak measurement approach.	No structured experimental evaluation or incorrect performance measurement.
Report Analysis & Discussion (25%)	Excellent analysis linking empirical results to theoretical complexity ($O(1)$, $O(k)$, $O(n)$). Clear discussion of scalability, dataset size effects, and Hash Map vs Trie behaviour. Insightful interpretation of performance trends and limitations.	Very good analysis with clear reference to theoretical complexity. Minor gaps in depth or clarity.	Good discussion of results with reference to complexity, though analysis may lack depth or precision.	Basic description of results with limited theoretical linkage. Mostly descriptive rather than analytical.	Minimal analysis. Limited understanding of algorithm complexity demonstrated.	Little or no meaningful analysis. No connection to theoretical complexity.
Code Quality & Structure (10%)	Clean, modular, well-documented code. Logical project structure with separation of models, data loading, datasets, and sorters. Meaningful naming conventions. No redundancy. High readability and maintainability.	Well-structured and readable code with good modularity. Minor inconsistencies or minor redundancy.	Generally clear structure with some repetition or minor organisation issues. Naming mostly appropriate.	Basic structure. Some poor naming, duplication, or limited modular design.	Disorganised structure. Poor naming conventions. Limited modularity.	Poorly structured, unreadable, or incomplete code. No meaningful organisation.